A Generalized Software Workflow for Unanchored Approximate MUB Optimization: A Case Study in Dimension Six

Abdul Fatah, Ian McLoughlin, and Saim Ghafoor

Atlantic Technological University, Ireland

The search for mutually unbiased bases (MUBs) in composite dimensions remains a notorious challenge in quantum information theory, particularly in dimension six. We present a generalized, parameter-driven mathematical software workflow designed to optimize unanchored approximate MUB configurations across arbitrary dimensions d, candidate base counts n, and multi-seed sweeps. The underlying optimizer parameterizes unitary bases via Lie-algebra exponentiation, supported by a custom, hardware-accelerated complex Taylor-series layer for GPU compatibility. As a rigorous proof-of-work, we apply this software to study the optimization landscape in dimension six for varying numbers of candidate bases ($n = 3$ to 6). The experiments reveal a structured transition: the workflow recovers exact configurations for $n = 3$ and $n = 4$, while configurations for $n \geq 5$ become globally defective. Double-precision arithmetic resolves multiple local minima in the landscape, whereas single-precision is susceptible to basin selection effects. Performance scaling benchmarks demonstrate a hardware-acceleration crossover around dimension ninety. These results demonstrate the utility of our generalized, reproducible workflow as a scalable framework for high-dimensional quantum combinatorial searches.

Abdul Fatah: abdul.fatah@research.atu.ie
Ian McLoughlin: ian.mcloughlin@atu.ie
Saim Ghafoor: saim.ghafoor@atu.ie

1 Introduction

Mutually unbiased bases (MUBs) are central objects in quantum information theory, combinatorial design, and finite-dimensional Hilbert space geometry. Two orthonormal bases B and C in $\mathbb{C}^d$ are mutually unbiased [1] if

$$|\langle b, c\rangle|^2 = \frac{1}{d} \quad \text{for all } b \in B,\ c \in C.$$

A collection of bases is mutually unbiased if every pair satisfies this condition. Such structures play a fundamental role in quantum state tomography, quantum cryptography [2], and operator theory.

In dimensions that are prime or powers of primes, complete sets of $d+1$ mutually unbiased bases are known to exist [3]. In composite dimensions [4], and in particular in $d = 6$, the existence of a complete set of seven MUBs remains a long-standing open problem [5, 6]. While numerous analytic and computational approaches have been explored [7], no definitive resolution is known.

This paper adopts a software-engineering and computational physics perspective. We present a generalized, parameter-driven mathematical software workflow designed to find and study approximate mutually unbiased basis (AMUB) configurations in arbitrary dimensions. The focus is on providing a scalable, reliable computational framework for exploring high-dimensional MUB landscapes.

We consider an unanchored formulation, in which all candidate bases are treated symmetrically and optimized simultaneously. This differs from anchored formulations that fix one basis in advance. Each candidate basis is parameterized as a unitary matrix using Lie-algebra exponentiation [8], ensuring that the unitary constraint is satisfied by construction. The optimization objective measures the total deviation from the mutually unbiased condition across all pairs of bases.

The methodology is implemented in Python using PyTorch, NumPy, and marimo notebooks, with structured data export for reproducibility. The workflow explicitly records basis matrices, pairwise overlaps, diagnostics, and metadata, enabling independent regeneration and analysis of all results.

To demonstrate and validate the workflow, we target the notorious dimension six ($d = 6$) case for varying numbers of candidate bases (n). Multi-seed campaigns are performed for $n = 3, 4, 5, 6$, allowing us to distinguish reproducible structural features from initialization-dependent effects. The experiments reveal a progression from exact configurations at small n to fully defective configurations at larger n. In particular, the results indicate that configurations with $n \geq 5$ candidate bases do not exhibit near-exact MUB pairs under the present workflow, while exactness is recovered for $n = 3$ and $n = 4$.

The paper also investigates the effect of numerical precision by comparing complex128 reference runs with complex64 reduced-precision runs, focusing on loss values, near-exact classifications, basin selection, and unitarity residuals. A double-precision reference implementation (`complex128`) is contrasted against a single-precision baseline (`complex64`) on both CPU and hardware-accelerated GPU backends. The results show that while the qualitative transition from partial exactness to global defect is stable across precisions, the specific error bounds, near-exact pair classifications, and optimization basin selection are highly sensitive to numerical floating-point precision.

Overall, this work contributes a generalized, reproducible computational framework for exploring AMUB configurations, together with extensive case-study evidence about the structure of the optimization landscape in dimension six.

2 Contributions

This paper makes the following contributions:

Generalized, reproducible computational workflow: We design and implement a fully parameterized, dimension-agnostic software workflow for optimizing approximate mutually unbiased bases in arbitrary dimensions. The workflow includes structured artifact generation (basis matrices, overlap tensors, diagnostics, optimization histories, and metadata) to support independent reproduction and expansion to higher-dimensional searches.

Unanchored AMUB formulation: We study a fully unanchored optimization problem in which all candidate bases are optimized simultaneously without fixing a reference basis. This formulation enables symmetric exploration of the configuration space and exposes structural patterns that are not imposed by construction.

Unitary-preserving parameterization: We employ a Lie-algebra-based parameterization $U_k = \exp(iH_k)$ with Hermitian generators, enforcing unitarity by construction. This separates the unitary constraint from the optimization objective and simplifies the numerical treatment.

Portable hardware-accelerated Taylor exponentiation: We implement a custom Taylor-series matrix exponentiation layer for complex matrices and bound its truncation error under the norm ranges observed in the experiments. This layer bypasses backend device limitations for complex matrix exponentials, enabling native acceleration on Apple Silicon GPUs (MPS) and NVIDIA GPUs (CUDA) with truncation error bounded below floating-point resolution for the generator norms observed in the reported runs.

Multi-seed analysis of a nonconvex landscape: We perform multi-seed optimization campaigns for $n = 3, 4, 5, 6$ in $d = 6$, allowing us to identify reproducible structures and distinct basins of attraction. The results demonstrate that different initializations may converge to qualitatively different configurations.

Numerical case-study of a structural transition: As a validation case study, our optimization campaigns provide reproducible numerical evidence for a transition in the $d = 6$ unanchored landscape: configurations with $n = 3$ and $n = 4$ exhibit exact MUB relations, while all observed configurations for $n = 5$ and $n = 6$ are fully defective under the present workflow. This transition is robustly observed across multiple independent runs.

Precision-aware analysis: We compare complex128 and complex64 implementations of the workflow. The results show that while the qualitative transition to fully defective configurations is stable across precisions, the classification of near-exact pairs and the sampling of optimization basins are sensitive to numerical precision.

Multi-dimensional benchmarks and performance crossover: We perform multi-dimensional scaling benchmark campaigns comparing CPU and GPU runtimes across dimensions. We empirically demonstrate a performance crossover point around dimension ninety, verifying that the parallelized GPU implementation successfully overcomes polynomial central processing unit scaling for high-dimensional searches.

Compact structural diagnostics and visualization: We introduce visualization strategies, including sorted pairwise-loss spectra and aggregate summary plots, that provide a concise representation of the defect structure across configurations and parameter regimes.

3 Mathematical Formulation

This section defines the approximate mutually unbiased basis problem studied in this paper. We work throughout in dimension $d = 6$ and consider collections of n candidate orthonormal bases in the complex Hilbert space $\mathbb{C}^6$ [9]. Each basis is represented by a unitary matrix [10] $U_k \in U(6)$, whose columns are the basis vectors.

3.1 Mutual Unbiasedness

Let $B_i = \{u_{i,1}, \ldots, u_{i,6}\}$ and $B_j = \{u_{j,1}, \ldots, u_{j,6}\}$ be two orthonormal bases of $\mathbb{C}^6$. The bases are mutually unbiased [3] if every vector in one basis has equal squared overlap with every vector in the other basis:

$$|\langle u_{i,a}, u_{j,b}\rangle|^2 = \frac{1}{6}, \qquad 1 \le a, b \le 6. \tag{1}$$

Equivalently, if the bases are represented by unitary matrices U_i and U_j, the cross-Gram matrix $U_i^\dagger U_j$ satisfies

$$|(U_i^\dagger U_j)_{ab}|^2 = \frac{1}{6}, \qquad 1 \le a, b \le 6, \tag{2}$$

that is, all entries of $U_i^\dagger U_j$ have magnitude $1/\sqrt{6}$. The matrix $U_i^\dagger U_j$ is unitary, while this condition constrains only its entrywise squared moduli. Equivalently,

$$|U_i^\dagger U_j|^2 = \frac{1}{6}\mathbf{1}_{6\times 6}, \tag{3}$$

where the absolute value and square are taken entrywise, and $\mathbf{1}_{6\times 6}$ denotes the 6×6 all-ones matrix.

For an exact collection of n mutually unbiased bases, condition (3) must hold for every pair $1 \le i < j \le n$. In this paper, we do not attempt to certify exact existence or nonexistence. Instead, we use a continuous loss function to study approximate configurations and the structure of low-loss basins found by numerical optimization.

3.2 Approximate MUB Loss

For a pair of candidate bases U_i and U_j, we define the pairwise approximate-MUB defect as

$$\ell_{ij} = \left\| |U_i^\dagger U_j|^2 - \frac{1}{6}\mathbf{1}_{6\times 6} \right\|_F^2. \tag{4}$$

The total n-basis loss is the sum of all pairwise defects:

$$\mathcal{L}_n = \sum_{1\le i<j\le n} \ell_{ij} = \sum_{1\le i<j\le n} \left\| |U_i^\dagger U_j|^2 - \frac{1}{6}\mathbf{1}_{6\times 6} \right\|_F^2. \tag{5}$$

Thus, $\mathcal{L}_n = 0$ corresponds to an exact mutually unbiased collection of n bases. In numerical experiments, nonzero values of $\mathcal{L}_n$ quantify the residual deviation from mutual unbiasedness.

The loss admits a natural pairwise decomposition. We retain the individual quantities ℓ_{ij} in all experiments because they reveal whether residual defect is localized in a small number of basis pairs or distributed across the whole configuration. This decomposition also allows the optimized configuration to be viewed as a weighted complete graph on n vertices, with vertices corresponding to bases and edge weights corresponding to pairwise defects.

3.3 Unanchored Optimization

The optimization problem studied here is unanchored. No basis is fixed in advance to the identity, Fourier basis, or any other canonical representative. Instead, all candidate bases $U_1, \ldots, U_n$ are optimized simultaneously.

The unanchored AMUB optimization problem is therefore

$$\min_{U_1,\ldots,U_n \in U(6)} \mathcal{L}_n(U_1, \ldots, U_n). \tag{6}$$

This formulation treats all candidate bases symmetrically. The objective is invariant under a common left unitary action,

$$(U_1, \ldots, U_n) \mapsto (WU_1, \ldots, WU_n), \qquad W \in U(6),$$

so the unanchored formulation retains a global unitary gauge freedom. In contrast, an anchored formulation fixes one basis, for example $U_1 = I$, and optimizes only the remaining bases. Anchoring is often used for classification and to fix unitary-equivalence freedoms [11, 12], but it imposes a preferred reference frame during optimization. The unanchored formulation used here avoids this explicit choice and allows empirically observed structural patterns, described here as hub-like or clustered pairwise defect arrangements, to emerge from the optimization itself. Such empirical structures are consistent with the known rigidity and nonextendability phenomena for MUB configurations in dimension six [13].

3.4 Lie-Algebra Parameterization of Unitary Bases

To enforce unitarity by construction, each candidate basis is parameterized using Lie-algebra exponentiation [8]. For each basis index k, we define

$$U_k = \exp(iH_k), \qquad H_k = H_k^\dagger. \tag{7}$$

Since H_k is Hermitian, iH_k is skew-Hermitian, and therefore $\exp(iH_k)$ is unitary in exact arithmetic. This uses the Lie group–Lie algebra relation for the unitary group. The representation is not unique, but it provides a convenient differentiable parameterization for unconstrained numerical optimization.

In the implementation, H_k is obtained from an unconstrained trainable complex matrix $A_k \in \mathbb{C}^{6\times 6}$ by Hermitian symmetrization. The anti-Hermitian component of A_k is discarded by the symmetrization, so this implementation uses a redundant unconstrained representation of the Hermitian generator:

$$H_k = \frac{A_k + A_k^\dagger}{2}. \tag{8}$$

The trainable variables are therefore the unconstrained matrices $A_1, \ldots, A_n$, while the matrices used in the loss are the unitary matrices generated by (7). This separates the unitary constraint from the AMUB objective: the optimizer does not need to enforce unitarity through penalty terms or projections.

Combining (7) and (8), the computational parameterization is

$$U_k(A_k) = \exp\left(i\frac{A_k + A_k^\dagger}{2}\right), \qquad k = 1, \ldots, n. \tag{9}$$

The resulting map $A_k \mapsto U_k(A_k)$ is differentiable, so gradients of the loss can be propagated through the matrix exponential using automatic differentiation [14].

The numerical optimization problem can then be written as

$$\min_{A_1,\ldots,A_n \in \mathbb{C}^{6\times 6}} \mathcal{L}_n\left(U_1(A_1), \ldots, U_n(A_n)\right). \tag{10}$$

3.5 Diagnostics Derived from the Loss

The total loss $\mathcal{L}_n$ provides a scalar measure of approximate mutual unbiasedness, but it does not by itself describe the structure of the residual defect. For this reason, the workflow records several pairwise diagnostics in addition to the aggregate objective value.

For each pair (i, j), we compute the overlap matrix

$$M_{ij} = |U_i^\dagger U_j|^2, \tag{11}$$

where the absolute value and square are taken entrywise. The corresponding pairwise loss is

$$\ell_{ij} = \left\| M_{ij} - \frac{1}{6}\mathbf{1}_{6\times 6} \right\|_F^2, \tag{12}$$

which is the same pairwise defect used in the total loss (5). We also record the maximum entrywise deviation

$$\delta_{ij} = \max_{1\le a,b\le 6} \left| (M_{ij})_{ab} - \frac{1}{6} \right|. \tag{13}$$

Given a tolerance $\tau > 0$, a pair (i, j) is classified as near-exact if

$$\delta_{ij} \le \tau. \tag{14}$$

In the complex128 reference experiments, the primary tolerance used in the reported summaries is $\tau = 10^{-6}$. For precision-sensitivity analysis, especially in complex64, near-exact counts are also recomputed using relaxed tolerances. This separates the numerical classification rule from the underlying recorded quantities ℓ_{ij} and δ_{ij}.

We also monitor numerical unitarity by computing, for each optimized basis,

$$\epsilon_k = \|U_k^\dagger U_k - I_6\|_F. \tag{15}$$

Although the Lie-exponential parameterization enforces unitarity in exact arithmetic, this diagnostic measures deviations introduced by finite-precision arithmetic and numerical matrix exponentiation. In the reported experiments, these residuals are used to verify that the observed AMUB defects are not artifacts of substantial loss of unitarity.

3.6 Interpretation as Pairwise Defect Geometry

The pairwise decomposition of $\mathcal{L}_n$ is central to the interpretation of the numerical results. Each optimized configuration determines a weighted complete graph with vertex set $\{1, \ldots, n\}$ and edge weights ℓ_{ij}. We refer to this weighted-graph representation as the pairwise defect geometry of the configuration. Near-zero edges correspond to near-exact MUB pairs, while nonzero edges indicate defective pairwise relations.

This graph viewpoint allows different structural regimes to be distinguished. For example, a configuration may contain one basis that forms near-exact pairs with several others, together with a localized defective triangle among the remaining bases. Alternatively, it may exhibit globally distributed defect, with every pair carrying nonzero loss. The results sections use this pairwise defect geometry to compare optimized configurations across different values of n and numerical precisions.

4 Software Workflow and Precision Policy

This section describes the computational workflow used to generate and analyze the approximate MUB configurations. The workflow is designed as a reproducible mathematical-software artifact rather than as a one-off numerical experiment. It records the optimization configuration, numerical precision, backend policy, basis matrices, overlap matrices, diagnostics, and summary tables in machine-readable formats. The experimental protocol built on this workflow is described in Section 5, and the resulting numerical evidence is reported in Section 6.

4.1 Implementation Overview

The implementation is written in Python and uses PyTorch for differentiable optimization, NumPy for array storage and post-processing, and marimo notebooks for executable computational narratives. The notebook interface is used for executable presentation, while the core experiment functions are parameterized and reusable outside the notebook environment. The workflow is organized around a single parameterized experiment function that accepts the dimension d, the number of candidate bases n, the numerical dtype, random seed, optimizer settings, and output directory.

For each experiment, the workflow performs the following steps:

1. construct an unanchored Lie-exponential AMUB model with n trainable bases in $\mathbb{C}^d$;
2. optimize the total pairwise AMUB loss using Adam;
3. record the best basis configuration encountered during optimization;
4. compute pairwise overlap diagnostics and unitarity residuals;
5. save basis matrices, overlap matrices, diagnostics, optimization histories, and run metadata;
6. aggregate results across seeds and precisions for later analysis.

This design allows the same code path to be used for single-run validation, multi-seed campaigns, and precision-comparison experiments. The implementation supports the full range $n = 2, \ldots, 7$ used in the exploratory sweep, while the multi-seed campaigns focus on $n = 3, \ldots, 6$.

4.2 Backend Policy and Hardware Acceleration via Taylor Series

The local reference platform used for the experiments is an Apple Silicon system. PyTorch [15] detected the Metal Performance Shaders (MPS) backend, which provides access to Apple GPU execution through the `mps` device. However, the Lie-algebra parameterization used in this workflow requires repeated evaluation of the complex

matrix exponential $\exp(iH_k)$. In the local PyTorch/MPS environment, the required complex `torch.matrix_exp` operation was not natively implemented on the MPS backend. To resolve this limitation and support portable execution on non-CUDA accelerator backends, we developed a customized Taylor-series matrix exponentiation layer:

$$T_N(X) = \sum_{m=0}^{N} \frac{X^m}{m!}, \qquad \exp(X) \approx T_N(X) \quad (16)$$

To quantify the truncation error of this approximation, we use the following spectral-norm bound. For a skew-Hermitian matrix $X = iH$, with $H = H^\dagger$, the truncation error of the order-N Taylor series $T_N(X)$ in the spectral norm is bounded by:

$$\| \exp(X) - T_N(X) \|_2 \leq \frac{\|X\|_2^{N+1}}{(N+1)!} e^{\|X\|_2} \quad (17)$$

For $X = iH$ with $H = H^\dagger$, we have $\|X\|_2 = \|H\|_2$. Although every unitary matrix admits a principal Hermitian logarithm with spectral norm at most π, the trainable generators in our parameterization are not explicitly constrained to this principal branch. Therefore, the workflow records generator spectral norms at saved checkpoints and at the best saved iterate, evaluating the truncation bound using the observed norms. This is the standard Taylor remainder estimate for the matrix exponential in a submultiplicative norm [16].

Across the reported multi-seed campaigns and precisions, the maximum generator norm observed at the best saved iterate was approximately $\|H_k\|_2 = 2.71$. With $N = 20$, the truncation bound evaluated at this observed maximum norm is

$$\frac{2.71^{21}}{21!} e^{2.71} \approx 3.64 \times 10^{-10}.$$

To guarantee precision bounds in generalized, higher-dimensional searches, the software dynamically monitors the Frobenius norm of the generator matrix. If the norm exceeds a threshold of 3.0, the workflow issues a warning suggesting that the Taylor order N be increased or the optimizer parameters adjusted. This bound concerns the mathematical Taylor truncation error only. Floating-point accumulation, matrix-multiplication roundoff, and backend-specific numerical effects are monitored through the recorded unitarity residuals and precision-comparison diagnostics. Because $T_N(iH)$ is not exactly unitary, Taylor-based GPU runs are interpreted as approximate exponential runs rather than exact Lie-exponential evaluations, and they are accompanied by the unitarity residual diagnostic $\|U_k^\dagger U_k - I_d\|_F$ defined in Section 3. Since this custom layer relies exclusively on dense matrix multiplication and addition, it bypasses the MPS complex matrix exponential limitation and enables execution on the Apple M1 GPU without CPU fallback.

In our workflow, the implementation automatically routes matrix exponentials based on the active device:

- On CPU, it uses PyTorch's native `torch.matrix_exp(1j * H)` as reference implementation.
- On GPU (`mps` or `cuda`), it dynamically employs the Taylor-series expansion layer to avoid `NotImplementedError` or CPU-fallback bottlenecks.

This backend policy enables running the identical unanchored AMUB optimization workflow across CPUs, Apple Silicon GPUs, and NVIDIA CUDA platforms, which we evaluate in Section 6.6.

4.3 Precision Policy

The workflow treats numerical precision as part of the experimental configuration. The complex128 implementation is used as the reference precision because the diagnostics involve small residuals, near-exact pair classifications, and unitarity checks. The workflow records sufficient diagnostics to test the stability of near-exact classifications under multiple tolerances.

A complex64 version of the same workflow is used as a precision-sensitivity experiment. The purpose of the complex64 campaign is not to replace the complex128 reference, but to examine whether the qualitative structure of the AMUB landscape is preserved under reduced complex precision. In particular, the workflow compares loss values, near-exact pair counts, basin diversity, and unitarity residuals across complex128 and complex64.

For near-exact pair classification, the primary tolerance is 10^{-6} in the reported summaries. Because complex64 arithmetic has lower numerical resolution, the workflow also recomputes near-exact counts using relaxed tolerances, including 10^{-5} and 10^{-4}, to distinguish genuine structural changes from tolerance-sensitive classifications.

4.4 Optimization Components

Each run constructs a model consisting of n trainable complex matrices $A_1, \ldots, A_n$. These matrices are converted to Hermitian generators by

$$H_k = \frac{A_k + A_k^\dagger}{2},$$

and each basis is generated by

$$U_k = \exp(iH_k).$$

The AMUB loss is then evaluated over all $\binom{n}{2}$ basis pairs.

The optimization uses the Adam optimizer with fixed hyperparameters recorded in the run metadata and summarized for complete reproducibility in Table 1 (see Section 5). During optimization, the workflow tracks both the current loss and the best loss encountered. The basis matrices corresponding to the best observed loss are retained for post-processing and artifact export.

The workflow does not claim that the resulting configurations are global minimizers. The objective is nonconvex, and different random seeds may converge to different basins. For this reason, multi-seed campaigns are used to identify recurrent structures and basin diversity.

4.5 Saved Artifacts

For each run, the workflow writes a self-contained artifact directory. The directory name encodes the dimension, number of bases, dtype, and seed. Each run directory contains:

- `best_bases.npy`, storing the optimized basis matrices;
- `pairwise_overlaps.npz`, storing all matrices $|U_i^\dagger U_j|^2$;
- `pairwise_diagnostics.json`, storing pairwise losses, maximum deviations, and overlap statistics;
- `unitarity_residuals.json`, storing the residuals $\|U_i^\dagger U_i - I\|_2$ to verify numerical orthonormality;
- `generator_norms.json`, storing norms of the Lie-algebra generator parameters;
- `optimization_history.csv`, storing the loss trace recorded during optimization;
- `run_metadata.json`, storing problem parameters, optimizer settings, dtype, backend policy, and diagnostic summaries.

Campaign-level summaries are also saved. These include per-run CSV files, aggregate JSON summaries by n and dtype, precision-comparison summaries, representative-run manifests, and figure-data JSON files. The artifact structure is intended to make it possible to inspect the reported results without rerunning the full optimization.

4.6 Visualization Workflow

The visualization stage is separated from the optimization stage. Rather than recomputing optimized bases, visualization routines load the saved overlap matrices and diagnostics from disk. This separation reduces ambiguity between computation and post-processing.

The workflow produces two types of visual summaries. First, aggregate plots show loss statistics and near-exact pair counts as functions of n and numerical precision. Second, sorted pairwise-loss spectra show the distribution of pairwise defects within representative configurations. These compact spectra are used in the main paper because full heatmap grids become large as n increases. Full pairwise overlap heatmaps are retained as supporting artifacts.

4.7 Software Generalization and Multi-Dimensional Scalability

Although the experiments in this paper focus on the longstanding open problem in $d = 6$, the PyTorch-based unanchored software architecture is fully generalized and scalable to arbitrary dimensions d and candidate basis counts n. A researcher can adapt the workflow to study approximate MUBs in $d = 7, 8$, or composite dimensions like $d = 10$ and $d = 12$ simply by passing the desired parameters to the core model instantiation:

```
model = UnanchoredAMUBModel(d=10, n_bases=8)
```

The same software architecture may be adapted to related unitary-constrained search problems, such as Quantum Latin Squares [17], Unitary Error Bases [18], and biunitary matrices [19], after replacing the AMUB loss with problem-specific constraint losses.

As the dimension increases, the dense matrix operations become more computationally intensive, and hardware acceleration is expected to become more favorable. The benchmark results in 6.6 quantify this trend for the tested dimensions and extrapolate the observed crossover behavior. The benchmark results suggest that, as dimension increases, the larger dense linear-algebra workload can amortize fixed accelerator-launch overheads, making GPU execution increasingly favorable for larger-dimensional searches such as larger-dimensional quantum combinatorial design searches.

5 Experimental Methodology

This section describes the numerical experiments used to evaluate the unanchored AMUB workflow in dimension six. The experiments are designed to study the structure of optimized approximate MUB configurations as the number of candidate bases increases, and to assess the sensitivity of the observed structures to random initialization and numerical precision.

5.1 Experimental Goals

The experiments address three questions.

First, we ask how the optimized AMUB configurations change as the number of candidate bases increases from small collections to the complete-set target size. The complete-set target $n = 7$ in $d = 6$is included as an exploratory single-seed case, while the main multi-seed analysis focuses on $n = 3, \ldots, 6$.

Second, we ask whether observed structures are robust to random initialization. Because the objective is nonconvex, a single run may not be representative of the optimization landscape. We therefore perform multi-seed campaigns for selected values of n.

Third, we ask whether the qualitative structures observed in the complex128 reference workflow persist under complex64 execution. This precision comparison is intended to distinguish robust structural features from precision-sensitive diagnostics or basin-selection effects.

5.2 Single-Seed Sweep over Candidate Basis Counts

As an initial validation and exploratory sweep, the workflow is run for $n = 2, \ldots, 7$ using the complex128 reference precision and a fixed random seed. This sweep serves two purposes. First, it verifies that the same parameterized implementation can handle all candidate-basis counts in the range $2 \leq n \leq 7$, from the basic two-basis case to the formal complete-set target $n = d+1 = 7$ in dimension $d = 6$. Second, it produces representative configurations and saved artifacts for preliminary inspection before the multi-seed campaigns.

The case $n = 2$ is included as a positive control, since an unbiased pair is known to be attainable and should correspond to a near-zero AMUB loss under successful optimization. The case $n = 7$ is included as an exploratory complete-target run, but it is not used by itself to draw statistical conclusions about the existence or nonexistence of complete MUB sets in dimension six.

For each value of n, the workflow constructs n trainable bases in $\mathbb{C}^6$, optimizes the AMUB loss, records the best configuration encountered during optimization, and saves the corresponding artifacts. The number of unordered pairwise AMUB constraints grows as

$$\binom{n}{2},$$

so the sweep also provides a basic computational check that the implementation scales consistently with the number of basis pairs.

The single-seed sweep is not used as a statistical characterization of the nonconvex optimization landscape. Instead, it serves as a validation and exploratory step that motivates the subsequent multi-seed campaigns, where initialization sensitivity, basin diversity, and precision effects are examined more systematically.

5.3 Multi-Seed Campaigns

To assess sensitivity to initialization, we perform multi-seed campaigns for $n = 3, 4, 5, 6$. These

values are selected because they cover the transition from exact or partially exact configurations to fully defective configurations in the observed landscape. The case $n = 2$ is used as a basic positive-control case and is not the focus of the multi-seed study. The case $n = 7$ is included in the single-seed sweep as the complete-set target, but the multi-seed campaigns reported here focus on $n = 3, \ldots, 6$.

For each n in the multi-seed campaign, the workflow runs 100 independent seeds:

$$s \in \{0, 1, \ldots, 99\}.$$

This larger-scale campaign is designed to provide a robust characterization of basin diversity and stability across random initializations. The workflow is designed to support arbitrary seed pools without changing the experimental code path.

For every pair (n, s), the model is reinitialized, optimized, diagnosed, and saved independently. The reported multi-seed summaries include the minimum, median, and maximum best loss over the 100 seeds, and the min/median/max near-exact MUB pairs observed.

5.4 Optimization Settings

All experiments use the same unanchored Lie-exponential parameterization described in Section 3. The trainable variables are unconstrained complex matrices A_k, which are symmetrized to Hermitian matrices H_k before exponentiation.

The optimizer is Adam. All important optimization and campaign-level hyperparameters are summarized in Table 1 to ensure full experimental reproducibility. The optimization history stores the current loss and best loss at regular logging intervals.

During optimization, the workflow tracks the best loss encountered rather than relying only on the final iterate. This is important because the objective is nonconvex and the Adam iterates may fluctuate within a basin. The basis matrices corresponding to the best observed loss are retained and used for all subsequent diagnostics and artifact generation.

As detailed in Table 1, the workflow uses a base step count of 1500 steps for smaller configurations ($n < 5$) and an increased step count of 2000 steps for harder cases with $n \geq 5$ bases to allow sufficient convergence in the higher-dimensional constraint landscapes.

5.5 Precision Campaigns

The complex128 workflow is treated as the reference implementation. This precision is used for the main structural analysis because the diagnostics include small unitarity residuals, near-exact pair classifications, and pairwise deviations from 1/6.

A corresponding complex64 campaign is run using the same parameterized workflow, the same values of n, and the same seed set for the multi-seed experiments. The purpose of the complex64 campaign is not to replace the complex128 reference results, but to evaluate whether the same qualitative structures persist under reduced complex precision.

To isolate numerical precision effects from Taylor truncation errors, the primary structural AMUB experiments and precision-comparison campaigns are executed entirely on the CPU backend using PyTorch's native `torch.matrix_exp` implementation. Unless otherwise stated, the precision-comparison campaigns use this reference CPU pathway, separating backend-specific Taylor GPU execution and timing experiments (Section 6.6) from the AMUB structural analysis.

The precision comparison focuses on:

- best loss statistics over seeds;
- number of near-exact basis pairs;
- basin diversity as indicated by rounded best losses;
- unitarity residuals;
- stability of the transition from partial exactness to fully defective configurations.

5.6 Near-Exact Pair Classification

For each optimized configuration, we compute the maximum entrywise deviation

$$\delta_{ij} = \max_{a,b} \left| \left(|U_i^\dagger U_j|^2 \right)_{ab} - \frac{1}{6} \right|$$

for every pair (i, j). A pair is classified as near-exact $\delta_{ij} \leq \tau$.

The primary tolerance used in the summary tables is

$$10^{-6}.$$

Table 1: Optimization and campaign-level hyperparameters used in the reported AMUB campaigns. The Taylor order applies only to GPU-backend timing experiments.

Hyperparameter	Value / Setting
Optimizer	Adam
Learning rate (η)	0.02
Initialization scale	0.05
Step count (for $n < 5$)	1500 steps
Step count (for $n \geq 5$)	2000 steps
Primary numerical precision	`complex128` (double-precision reference)
Secondary numerical precision	`complex64` (single-precision baseline)
Campaign seed set (s)	$\{0, 1, 2, 3, 4\}$
Single-seed validation seed (s)	1234
Primary near-exact tolerance (τ)	10^{-6}
Taylor expansion order (N)	20 (utilized on GPU backends)

For precision-sensitivity analysis, particularly for complex64 runs, the near-exact counts are also recomputed using relaxed tolerances

$$10^{-5} \quad \text{and} \quad 10^{-4}.$$

This additional check is used to determine whether changes in near-exact pair counts reflect genuine structural differences or tolerance-sensitive numerical effects.

5.7 Pairwise Structural Diagnostics

The scalar loss $\mathcal{L}_n$ is not sufficient to characterize the geometry of an optimized configuration. Therefore, the workflow records pairwise diagnostics for every pair (i, j), including:

- pairwise loss contribution ℓ_{ij};
- minimum, maximum, mean, and standard deviation of $|U_i^\dagger U_j|^2$;
- maximum absolute deviation δ_{ij} from $1/6$;
- near-exact classification under the selected tolerance.

The pairwise losses are used to identify structural patterns such as defective triangles, hub-and-triangle configurations, and globally distributed defect. Here a hub-and-triangle configuration denotes one basis forming near-exact pairs with several others, while the remaining bases form a localized defective triangle. For visualization, the pairwise losses are sorted from largest to smallest, producing a compact spectrum of pairwise defects for each representative configuration.

5.8 Representative Configurations

After the multi-seed campaigns, representative runs are selected for visualization. The representative selection is based on the saved run summaries and diagnostics rather than manual inspection of heatmaps. Within each structural category, the lowest-loss run is selected as the representative. If multiple runs have the same rounded loss, the smallest seed index is used as a deterministic tie-breaker.

The representative set includes, when present:

- an exact or near-exact $n = 3$ configuration;
- a defective $n = 3$ triangular configuration;
- an $n = 4$ hub-and-triangle configuration;
- a best observed fully defective $n = 5$ configuration;
- a best observed $n = 6$ low-loss configuration;
- a recurrent $n = 6$ basin when available.

These representatives are used to generate sorted pairwise-loss spectra and supporting heatmap visualizations. The representative-run manifest records the selected run directory, seed, dtype, best loss, number of near-exact pairs, and paths to the saved diagnostics and overlap matrices.

5.9 Visualization and Figure Construction

The main paper uses compact visual summaries rather than full heatmap grids for every configu-

ration. Three types of figures are generated from saved artifacts:

1. loss summaries as a function of n and dtype, with min–median–max ranges over seeds;
2. near-exact pair count summaries as a function of n and dtype;
3. sorted pairwise-loss spectra for representative configurations.

Full pairwise heatmap grids are generated as supporting artifacts. These heatmaps visualize the matrices $|U_i^\dagger U_j|^2$ directly, but they become large as n increases. For example, the complete-target case $n = 7$ contains $\binom{7}{2} = 21$ pairwise overlap matrices. For this reason, the main text emphasizes compact quantitative summaries and pairwise-loss spectra. For each plotted figure, the numerical data used to generate the figure are saved separately from the rendered image, allowing figures to be regenerated without rerunning optimization.

5.10 Scope and Limitations of the Experiments

The experiments are numerical and do not constitute a proof of existence or nonexistence of complete MUB sets in dimension six. The optimizer is not guaranteed to find global minima, and the observed basins depend on initialization, precision, and optimizer settings.

The multi-seed campaigns reported here use 100 independent seeds per value of n for $n = 3, \ldots, 6$. This large-scale sweep provides a robust statistical characterization of the full nonconvex landscape and its basins.

The complex128 results are used as the primary reference because they provide stable near-exact classifications and small unitarity residuals. The complex64 results are interpreted as a precision-sensitivity study. Differences between complex128 and complex64 are therefore treated as evidence of precision-dependent optimization behavior rather than as a ranking of one precision as universally superior.

The reported landscape is also conditioned on the Lie-exponential parameterization and Adam optimizer; alternative manifold-optimization methods or parameterizations may sample different basins.

6 Results

This section reports the numerical results obtained from the unanchored AMUB workflow in dimension six. We first summarize the single-seed sweep over $n = 2, \ldots, 7$, then present the multi-seed complex128 campaign for $n = 3, \ldots, 6$, followed by the complex64 precision-sensitivity campaign. Finally, we interpret the pairwise defect spectra of representative configurations.

The results are numerical and should be interpreted as evidence about the optimization landscape explored by the workflow. They do not constitute a proof of existence or nonexistence of exact MUB configurations in dimension six.

6.1 Single-Seed Sweep over $n = 2, \ldots, 7$

As an initial validation and exploratory study, we ran the complex128 reference workflow for $n = 2, \ldots, 7$ using a fixed seed. The results are summarized in Table 2. This sweep verifies that the same parameterized workflow applies uniformly across the range from a single MUB pair to the complete-set target size $n = 7$.

For $n = 2$, the workflow recovered a mutually unbiased pair to numerical precision, with loss approximately 1.54×10^{-30}. This provides a positive control for the loss function and the Lie-exponential parameterization. For $n = 3$, the fixed-seed run reached a nonzero defective triangular configuration with total loss approximately 0.051249218996 and no near-exact pairs. For $n = 4$, the total loss was again approximately 0.051249218996, but the pairwise structure differed: three pairs were near-exact and three pairs were defective. This equality of total loss between the fixed-seed $n = 3$ and $n = 4$ runs is explained by the pairwise decomposition: the $n = 4$ configuration contains a defective triangular component together with three near-exact hub edges.

For $n = 5$, the fixed-seed run produced a fully defective configuration with no near-exact pairs and total loss approximately 0.359638850581. The $n = 6$ fixed-seed run reached a recurrent nonzero basin near 0.965932502216, while the $n = 7$ fixed-seed run produced a fully defective complete-target configuration with loss approximately 1.584472133131. In both cases, every pairwise relation was defective under the primary near-exact tolerance.

The single-seed sweep suggests a progression

Table 2: Single-seed complex128 sweep over $n = 2, \ldots, 7$ in dimension six. Near-exact pairs are counted using the tolerance 10^{-6}.

n	Number of pairs	Best loss	Near-exact pairs	Defective pairs
2	1	1.543×10^{-30}	1	0
3	3	0.051249218996	0	3
4	6	0.051249218996	3	3
5	10	0.359638850581	0	10
6	15	0.965932502216	0	15
7	21	1.584472133131	0	21

from exact behavior at $n = 2$, through partially exact or structured defective configurations at $n = 3$ and $n = 4$, to fully defective configurations for $n \geq 5$. The following multi-seed campaigns test which of these features persist across random initialization.

6.2 Multi-Seed complex128 Campaign

The complex128 campaign used 100 independent seeds for each $n = 3, 4, 5, 6$. Table 3 summarizes the loss statistics and near-exact pair counts observed in the campaign.

For $n = 3$, the workflow reached both exact and defective basins. A majority of seeds produced configurations with three near-exact pairs and loss numerically indistinguishable from zero at the reported scale (with median loss $\sim 10^{-29}$). However, some seeds produced fully defective configurations, including the defective triangular basin near 0.051249218996 and another defective basin near 0.065592. Thus, even when exact triples are reachable by the workflow, the unanchored nonconvex landscape contains competing defective attractors.

For $n = 4$, a significant fraction of seeds converged to the same low-loss basin near 0.051249218996, with three near-exact pairs and three defective pairs. This is the hub-and-triangle structure identified in the single-seed analysis. The remaining seeds reached higher-loss fully defective basins. The repeated recovery of the 0.051249218996 basin suggests that the hub-and-triangle configuration is a prominent attractor in the complex128 workflow.

For $n = 5$, all 100 seeds produced fully defective configurations with no near-exact pairs. The best loss observed was approximately 0.359637, while the median and maximum best losses were approximately 0.439033 and 0.689537, respectively. Most importantly, no near-exact pair was observed in any of the 100 runs.

For $n = 6$, all 100 seeds were also fully defective. The campaign sampled multiple nonzero basins, including losses near 0.868532, 0.965933, and higher. The best observed complex128 loss in this campaign was 0.868532, and the median best loss was also 0.868532. These results strongly support the empirical evidence that the six-basis landscape sampled by the present unanchored workflow is fully defective under the primary near-exact tolerance.

The complex128 campaign supports a transition from exact or partially exact configurations for $n = 3$ and $n = 4$ to fully defective configurations for $n = 5$ and $n = 6$. In this sense, the observed transition begins at $n = 5$ in the present workflow.

6.3 Precision Sensitivity: complex128 versus complex64

We repeated the multi-seed campaign for $n = 3, 4, 5, 6$ using complex64 arithmetic with 100 independent random seeds. Table 4 compares the complex128 and complex64 summaries. As described in Section 5, these structural precision comparisons use the CPU matrix-exponential pathway rather than the Taylor GPU backend.

For $n = 3$, both precisions found exact and defective basins. In the complex64 campaign, a majority of seeds reached exact or near-exact triples, while others reached the defective triangular basin. Thus, the coexistence of exact and defective basins is observed in both precisions.

For $n = 4$, the complex64 campaign found low-loss configurations near 0.051249, but the near-exact classification was more sensitive to tolerance than in complex128. Under the primary tolerance 10^{-6}, the median near-exact count for

Table 3: Multi-seed complex128 campaign for $n = 3, \ldots, 6$. The near-exact counts list the min/median/max number of near-exact pairs observed across 100 random seeds, using tolerance 10^{-6}.

n	Seeds	Loss min/median/max	Near-exact min/med/max
3	100	0.000000/0.000000/0.134080	0/3/3
4	100	0.051249/0.051249/0.294147	0/3/3
5	100	0.359637/0.439033/0.689537	0/0/0
6	100	0.868532/0.868532/1.404517	0/0/0

complex64 was 0, while the maximum was 3. A subsequent tolerance check showed that some complex64 $n = 4$ runs recover three near-exact pairs only under relaxed tolerances such as 10^{-5} or 10^{-4}. By contrast, the complex128 $n = 4$ hub-and-triangle classifications were stable under the tolerances examined.

For $n = 5$, both precisions produced fully defective configurations in all 100 seeds. This is the strongest precision-stable transition result: no near-exact pairs were observed for $n = 5$ under either complex128 or complex64, even when relaxed near-exact tolerances were examined.

For $n = 6$, both precisions produced fully defective configurations. However, the observed basin sampling differed. Reduced precision can affect basin selection and attractor stability in the nonconvex AMUB landscape, while preserving the qualitative fully defective structure for $n = 6$.

The precision comparison suggests that the qualitative transition from partial exactness to fully defective configurations is stable across complex128 and complex64. At the same time, the detailed basin frequencies and some near-exact classifications are precision-sensitive.

6.4 Tolerance Sensitivity of Near-Exact Classification

Near-exact pair classification depends on a numerical threshold. The primary threshold used in the summary tables is 10^{-6}, but additional checks were performed at 10^{-5} and 10^{-4}.

For the complex128 campaign, the near-exact counts were unchanged across these tolerances. Exact or near-exact configurations remained classified as such, and fully defective configurations for $n = 5$ and $n = 6$ remained fully defective.

For the complex64 campaign, the main tolerance sensitivity occurred at $n = 4$. Some runs classified as having zero near-exact pairs under 10^{-6} exhibited three near-exact pairs under 10^{-5} or 10^{-4}. This indicates that reduced precision can affect the numerical classification of near-exact hub edges. However, the $n = 5$ and $n = 6$ complex64 runs remained fully defective under all examined tolerances. Therefore, within the examined tolerance range, the observed disappearance of near-exact pairs for $n \geq 5$ is not an artifact of the strict 10^{-6} threshold in the present experiments.

6.5 Pairwise Defect Spectra

The pairwise defect spectra provide a compact visualization of the structure hidden behind the scalar loss values. Each spectrum consists of the sorted pairwise losses ℓ_{ij} for a representative optimized configuration. The sorted pairwise-loss spectra for representative complex128 and complex64 configurations are shown in Appendix Figures 5 and 6, respectively.

The $n = 3$ exact representative has all pairwise losses at numerical zero. The $n = 3$ defective representative has three approximately equal nonzero pairwise losses, forming a symmetric defective triangle.

The $n = 4$ hub-and-triangle representative has three near-zero pairwise losses and three approximately equal defective losses. This supports the interpretation that the $n = 4$ low-loss basin consists of a defective three-basis core together with a near-exact hub basis.

For $n = 5$, all ten pairwise losses are nonzero in the representative best configuration. The spectrum is not uniform: one pair carries substantially more defect than the others in the selected representative. This indicates that the transition at $n = 5$ is not merely a uniform spreading of the $n = 4$ defect, but a reorganization into a fully defective configuration.

For $n = 6$, the representative low-loss basin has all fifteen pairwise losses nonzero, with repeated

Table 4: Precision comparison for multi-seed AMUB campaigns in dimension six. Near-exact min/med/max use the primary tolerance 10^{-6} across 100 independent seeds.

n	Dtype	Seeds	Loss min/median/max	Near-exact min/med/max
3	complex128	100	0.000000/0.000000/0.134080	0/3/3
3	complex64	100	0.000000/0.000000/0.111767	0/3/3
4	complex128	100	0.051249/0.051249/0.294147	0/3/3
4	complex64	100	0.051249/0.051249/0.409379	0/0/3
5	complex128	100	0.359637/0.439033/0.689537	0/0/0
5	complex64	100	0.359636/0.439033/0.772279	0/0/0
6	complex128	100	0.868532/0.868532/1.404517	0/0/0
6	complex64	100	0.868532/0.868532/1.493699	0/0/0

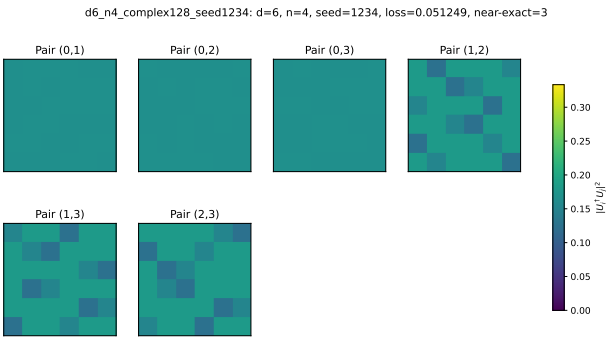

Figure 1: Representative pairwise-loss heatmap for $d = 6, n = 4$. The heatmap shows a hub-and-triangle structure with three near-zero losses and three nonzero losses.

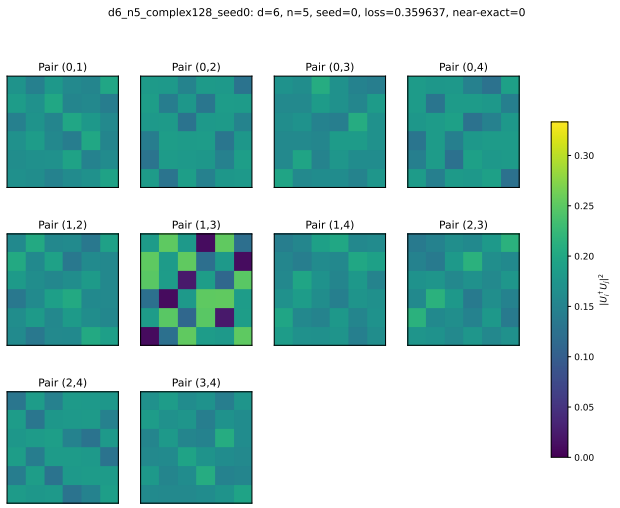

Figure 2: Representative pairwise-loss heatmap for $d = 6, n = 5$. The heatmap shows a fully defective structure with all pairs having nonzero loss.

loss levels. This is consistent with a globally defective but structured configuration, in which no pair is near-exact under the primary tolerance. The sorted spectra therefore support the interpretation that increasing n changes the optimized configurations from exact or partially exact pairwise relations to globally distributed defect.

6.6 Hardware Benchmarks and GPU Acceleration Performance

To evaluate the portable hardware-acceleration layer described in Section 4, we performed benchmark experiments on an Apple Silicon MacBook Pro (M1 processor) running PyTorch 2.4. We measured the average execution time (in seconds) required to perform 1000 optimization iterations for the case $n = 6$ bases, comparing CPU execution under reference `complex128` and baseline `complex64` precisions against the custom Taylor-series matrix exponential running natively on the Apple Silicon GPU via the `mps` backend. The results are summarized in Table 5.

Relative GPU speed is computed as CPU complex128 time divided by MPS GPU time; values above 1 indicate GPU speedup relative to the double-precision CPU baseline.

The benchmark results in Table 5 show a hardware crossover behavior typical of accelerator-backed dense linear algebra. For small dimensions, the Apple M1 GPU backend is substantially slower than the CPU baselines. At $d = 6$, the arithmetic workload is too small to amortize accelerator overheads, and the CPU completes 1000 iterations in under three seconds, while the MPS backend requires approximately 43 seconds.

As d increases, the CPU timings grow rapidly, reflecting the increasing cost of dense matrix products and matrix exponentiation. By contrast, the measured MPS timings remain in the range of approximately 43–48 seconds across the

Table 5: Multi-dimensional hardware benchmark and iteration speed comparison (in seconds) per 1000 optimization steps ($n = 6$ bases) across dimensions $d = 6, 12, 24, 48, 96$ on CPU vs. Apple M1 GPU (MPS). Order-20 Taylor expansion is utilized for the GPU backend.

Dimension d	CPU `c128` (s)	CPU `c64` (s)	M1 GPU MPS (s)	Relative GPU Speed
6	2.617	2.695	42.958	0.06×
12	3.367	3.099	44.105	0.08×
24	4.547	4.032	43.345	0.10×
48	13.077	9.782	45.735	0.29×
96	62.281	37.244	48.384	1.29×

tested dimensions. This nearly constant GPU timing is consistent with fixed accelerator-launch overheads dominating the small- and medium-dimensional workloads.

At $d = 96$, the MPS Taylor backend completes 1000 iterations in 48.384 seconds, compared with 62.281 seconds for the CPU complex128 reference timing. Thus, the measured timings show a crossover between $d = 48$ and $d = 96$ relative to the double-precision CPU baseline. Interpolating these timings suggests a crossover near $d \approx 80$. The MPS backend remains slower than the CPU complex64 timing in this benchmark, so the observed speedup should be interpreted specifically relative to the complex128 CPU reference.

These results indicate that the Taylor-series layer enables portable accelerator execution and can become favorable at larger dimensions, while preserving a CPU-native reference pathway for the structural AMUB experiments reported above.

To evaluate the scalability of the approach on enterprise-grade hardware, we additionally performed benchmark experiments on the JANUS high-performance computing cluster, which features AMD EPYC 7352 CPUs and NVIDIA A100 GPUs. As shown in Table 6, the CPU baseline times grow significantly with dimension, while the NVIDIA A100 GPU maintains a nearly constant timing of approximately 26–28 seconds across all dimensions up to $d = 96$. The hardware crossover occurs earlier on the JANUS cluster, overtaking the double-precision CPU baseline between $d = 24$ and $d = 48$, and overtaking the single-precision CPU baseline at $d = 96$. This confirms that the Taylor-series matrix exponential fallback delivers substantial acceleration for moderate-to-large tensor dimensions on high-performance accelerators.

Relative GPU speed is computed as CPU complex128 time divided by GPU complex128 time.

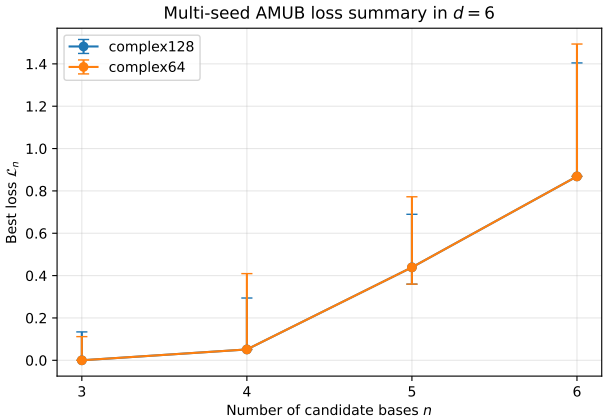

Figure 3: Best AMUB loss summary in $d = 6$. This plot shows the median best loss with min–max range across 100 seeds. The AMUB loss increases with n.

7 Conclusion

This paper presented a generalized, reproducible mathematical software workflow for unanchored approximate MUB optimization. The workflow is dimension-agnostic, representing candidate bases by Lie-exponential unitary parameterizations, optimizing all bases simultaneously without fixing a reference basis, and recording machine-readable artifacts. As a validation case study, we successfully applied the framework to dimension six, recovering exact configurations for small basis counts and demonstrating a robust structural transition to fully defective configurations for five or more bases.

The numerical experiments provide evidence, under the specified optimization protocol, for a transition in the observed unanchored AMUB landscape. Exact or partially exact configurations are recovered for smaller numbers of bases,

Table 6: Multi-dimensional hardware benchmark and iteration speed comparison (in seconds) per 1000 optimization steps ($n = 6$ bases) across dimensions $d = 6, 12, 24, 48, 96$ on the JANUS HPC cluster (AMD EPYC 7352 CPU vs. NVIDIA A100 GPU). Order-20 Taylor expansion is utilized for the GPU backend.

Dimension d	CPU `c128` (s)	CPU `c64` (s)	A100 GPU (s)	Relative GPU Speed
6	7.270	7.649	26.495	0.27×
12	10.314	10.870	27.317	0.38×
24	22.282	15.492	27.453	0.81×
48	31.449	22.983	27.578	1.14×
96	72.810	55.595	27.705	2.63×

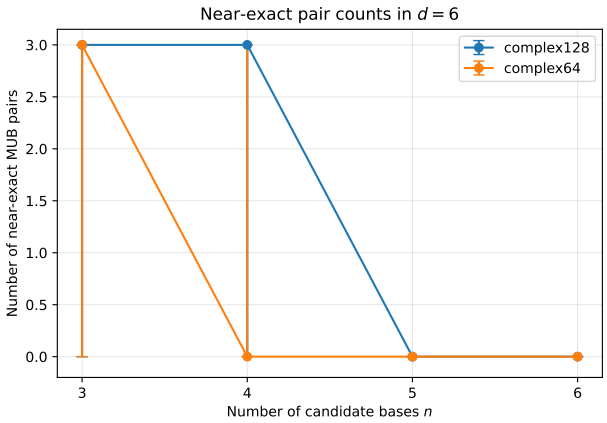

Figure 4: Near-exact MUB pair counts in $d = 6$. This plot shows the median number of near-exact MUB pairs with corresponding ranges across 100 seeds. Near-exact pairs disappear for $n \geq 5$ under the primary tolerance, coinciding with a transition to fully defective observed configurations.

including exact triples and four-basis hub-and-triangle configurations. In contrast, the reported campaigns for $n = 5$ and $n = 6$ produce fully defective configurations under the examined near-exact tolerances. Pairwise defect spectra show that this transition is not only a change in scalar loss, but also a change in the structure of the residual pairwise defects.

The precision comparison shows that the qualitative transition to fully defective configurations for $n \geq 5$ is stable across complex128 and complex64 in the reported campaigns, while basin selection and some near-exact classifications remain precision-sensitive. The hardware benchmarks further show that the Taylor-series matrix-exponential layer enables portable accelerator execution and becomes favorable relative to the CPU complex128 reference at larger tested dimensions.

The results are numerical and do not constitute a proof of existence or nonexistence of complete MUB sets in dimension six. They are conditioned on the Lie-exponential parameterization, Adam optimizer, selected tolerances, and finite seed campaigns. Future work includes larger seed sweeps, alternative manifold-optimization methods, extended precision studies, and broader searches in other composite dimensions

Acknowledgments

This work was supported by Atlantic Technological University, Ireland, under the Postgraduate Research Training Programme in Modelling and Computation for Health and Society (MOCHAS-PRTP). We also acknowledge the ATU JANUS High-Performance Computing (HPC) cluster facility for providing the computational resources required for the multi-seed campaigns.

References

[1] Andreas Klappenecker and Martin Rötteler. "Constructions of mutually unbiased bases". In Finite Fields and Applications: 7th International Conference, Fq7, Toulouse, France, May 5-9, 2003. Revised Papers. Pages 137–144. Springer (2004).

[2] Charles H. Bennett and Gilles Brassard. "Quantum cryptography: Public key distribution and coin tossing". Theoretical Computer Science **560**, 7–11 (2014).

[3] William K Wootters and Brian D Fields. "Optimal state-determination by mutually unbiased measurements". Annals of Physics **191**, 363–381 (1989).

[4] Daniel McNulty and Stefan Weigert. "Mutually Unbiased Bases in Composite Dimensions - A Review". Quantum **10**, 2051 (2026).

[5] Ingemar Bengtsson, Wojciech Bruzda, Åsa Ericsson, Jan-Åke Larsson, Wojciech Tadej, and Karol Życzkowski. "Mutually unbiased bases and hadamard matrices of order six". Journal of Mathematical Physics **48**, 052106 (2007).

[6] Paweł Horodecki, Łukasz Rudnicki, and Karol Życzkowski. "Five open problems in quantum information theory". PRX Quantum **3**, 010101 (2022).

[7] Stephen Brierley, Stefan Weigert, and Ingemar Bengtsson. "All mutually unbiased bases in dimensions two to five". Quantum Information & Computation **10**, 803–820 (2010).

[8] Elena Celledoni and Arieh Iserles. "Approximating the exponential from a lie algebra to a lie group". Math. Comput. **69**, 1457–1480 (2000).

[9] Gilbert Helmberg. "Introduction to spectral theory in hilbert space". Courier Dover Publications. (2008). url: https://www.sciencedirect.com/science/book/9780720423563.

[10] Dudley Ernest Littlewood. "The theory of group characters and matrix representations of groups". Volume 357. American Mathematical Soc. (1977).

[11] THOMAS DURT, BERTHOLD-GEORG ENGLERT, INGEMAR BENGTSSON, and KAROL ŻYCZKOWSKI. "On mutually unbiased bases". International Journal of Quantum Information **08**, 535–640 (2010).

[12] Stephen Brierley. "Mutually unbiased bases in low dimensions". PhD thesis. University of York. (2009). url: https://etheses.whiterose.ac.uk/id/eprint/587/.

[13] Markus Grassl. "On SIC-POVMs and MUBs in Dimension 6" (2004). arXiv:quant-ph/0406175.

[14] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. "Automatic differentiation in pytorch". In NIPS 2017 Workshop on Autodiff. (2017). url: https://openreview.net/forum?id=BJJsrmfCZ.

[15] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. "Pytorch: An imperative style, high-performance deep learning library". In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, editors, Advances in Neural Information Processing Systems 32. Pages 8024–8035. Curran Associates, Inc. (2019).

[16] Nicholas J. Higham. "Functions of matrices". Society for Industrial and Applied Mathematics. (2008).

[17] B. J. Musto. "Quantum latin squares and quantum functions: Applications in quantum information". PhD thesis. University of Oxford. (2019). url: https://www.cs.ox.ac.uk/people/bob.coecke/BenMustoThesis.

[18] Emanuel Knill. "Non-binary unitary error bases and quantum codes" (1996). arXiv:quant-ph/9608048.

[19] David J. Reutter and Jamie Vicary. "Biunitary constructions in quantum information". Higher Structures **3**, 109–154 (2019).

8 Appendix: Additional Figures

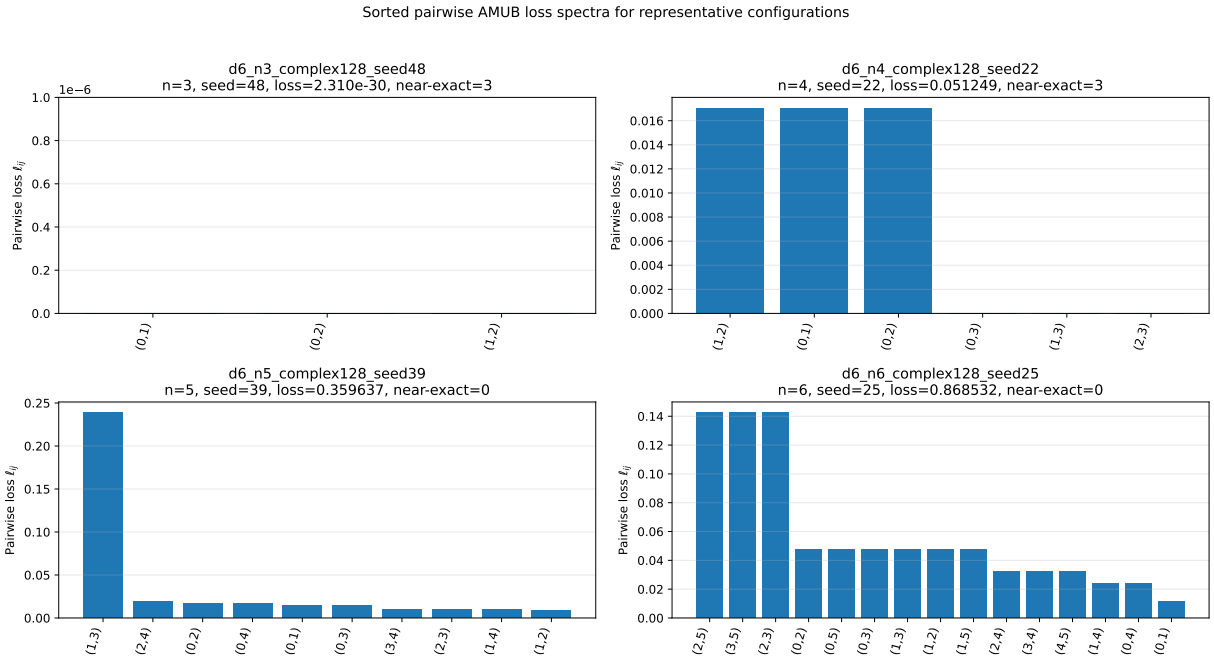

Figure 5: Sorted pairwise AMUB loss spectra for representative complex128 configurations. Each bar corresponds to one pairwise contribution ℓ_{ij}. The exact $n = 3$ representative has pairwise losses at numerical zero, the defective $n = 3$ representative has three approximately equal nonzero losses, the $n = 4$ representative exhibits three near-zero losses and three equal defective losses, and the $n = 5$ and $n = 6$ representatives have no zero pairwise losses.

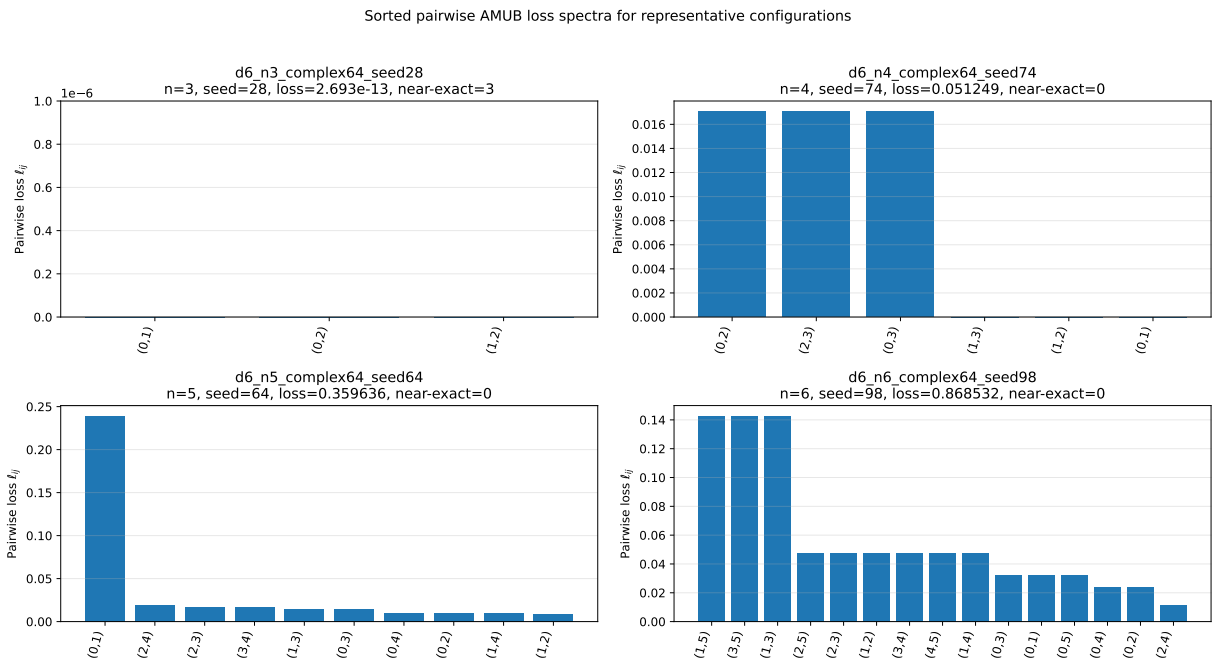

Figure 6: Sorted pairwise AMUB loss spectra for representative complex64 configurations. Machine-zero losses below the display tolerance are shown as zero for visualization only; raw values are retained in the saved figure-data artifact. The figure illustrates that the main transition to fully defective configurations for $n = 5$ and $n = 6$ persists in complex64, while the sampled basins and near-exact classifications differ from the complex128 reference campaign.